

# Revis Cloud Setup & Deployment Guide

A beginner-friendly walkthrough of every step taken to get  
your music collaboration platform live on the internet.

---

Completed: March 25, 2026

App URL: <https://revis-cloud-production.up.railway.app>

# What's In This Guide

1. **The Big Picture** — What Revis Cloud is and how the pieces fit together
2. **Clerk (Authentication)** — How users log in to your app
3. **Cloudflare R2 (File Storage)** — Where your audio files are stored
4. **Railway (Hosting)** — How your app runs on the internet
5. **Connecting Everything** — Environment variables and redirect URLs
6. **Quick Reference** — Dashboard links and what to do next

# 1. The Big Picture

---

Revis Cloud is a music collaboration platform — a website where you can upload tracks, manage versions, add stems, and share music with others. To make it work on the internet, we needed three separate services, each handling a different job:

| Service       | What It Does                                                                                                        |
|---------------|---------------------------------------------------------------------------------------------------------------------|
| Clerk         | Handles user accounts — signing up, logging in, logging out. Think of it like the bouncer at the door.              |
| Cloudflare R2 | Stores your audio files (tracks, versions, stems) in the cloud. Like a giant hard drive in the sky.                 |
| Railway       | Runs your app's code and makes it accessible at a URL anyone can visit. It's the actual computer your app lives on. |

## How They Work Together

When someone visits your app, here's what happens behind the scenes:

- **User visits** revis-cloud-production.up.railway.app → Railway serves the web page
- **User signs in** → Clerk verifies their identity and gives them a secure token (like a wristband at a concert)
- **User uploads a track** → the file goes to your Railway server, which checks the wristband, then sends the file to R2 for storage
- **User plays a track** → Railway generates a temporary, secure link to the file on R2, and the browser streams it directly from R2

**Plain English:** Your app is like a restaurant. Railway is the building, Clerk is the host who checks reservations, and R2 is the kitchen storage where all the ingredients (audio files) are kept.

## 2. Clerk (Authentication)

---

### What Is Clerk?

Clerk is a service that handles everything related to user accounts. Instead of writing thousands of lines of code to manage passwords, email verification, forgot-password flows, and security, you use Clerk and it does all of that for you. It's like hiring a security company instead of building your own alarm system.

### What We Did

#### Step 1: Created a Clerk account and app

We signed up at [dashboard.clerk.com](https://dashboard.clerk.com) and created an application called **Revis-Cloud**. Clerk gave us two keys:

| Key                    | What It Is                                                                                                         |
|------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Publishable Key</b> | A public key (starts with pk_test_...) that your website uses to show the login form. It's safe for anyone to see. |
| <b>Secret Key</b>      | A private key (starts with sk_test_...) that only your server uses behind the scenes. Never share this!            |

**Plain English:** Think of the publishable key as your business's phone number (public, on your website) and the secret key as the password to your bank account (private, never shared).

#### Step 2: Updated the code to use Clerk

The app needed to know its Clerk publishable key to show login forms. Since the key is stored as an environment variable on the server (more on that later), we created a small API endpoint — basically a URL the app can call to ask the server "what's the Clerk key?"

We added `/api/clerk-key` to the server, and updated three HTML files (`index.html`, `sign-in.html`, `sign-up.html`) to fetch the key from there when they load.

**Plain English:** An API endpoint is like a phone extension. Your app "calls" `/api/clerk-key` and the server "picks up" and reads back the key. An environment variable is a setting stored on the server — like a sticky note only the server can read.

#### Step 3: Configured redirect URLs

We told Clerk where your sign-in and sign-up pages live, so after a user logs in, Clerk knows to send them back to your app (not to some random page). We set:

| Setting        | Value                                         |
|----------------|-----------------------------------------------|
| Fallback host  | https://revis-cloud-production.up.railway.app |
| Sign-in path   | /sign-in                                      |
| Sign-up path   | /sign-up                                      |
| After sign-out | /sign-in (sends users back to login)          |

**Plain English:** A redirect URL is like a "return address" on a letter. When Clerk is done processing a login, it needs to know where to send the user back to.

## 3. Cloudflare R2 (File Storage)

---

### What Is Cloudflare R2?

R2 is cloud storage for files — think of it as a Dropbox or Google Drive, but designed for apps instead of people. Your audio files (tracks, versions, stems) are too big to store in a database, so they go here instead. The big advantage of R2 is that it has **zero egress fees**, meaning you don't pay extra when users listen to (download) your audio files.

***Plain English:** Most cloud storage charges you every time someone downloads a file. That would get very expensive for a music app where people stream audio constantly. R2 doesn't charge for downloads, only for storage.*

### Key Terms

| Term                 | What It Means                                                                                                                         |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Bucket</b>        | A container for files, like a folder. Ours is called "revis-audio".                                                                   |
| <b>Object</b>        | A single file stored in a bucket (an audio track, for example).                                                                       |
| <b>Presigned URL</b> | A temporary, secure link to a file that expires after a set time. This is how users stream audio without needing direct access to R2. |
| <b>API Token</b>     | A set of credentials (like a username + password) that lets your server talk to R2.                                                   |
| <b>S3-compatible</b> | R2 speaks the same "language" as Amazon S3, the most popular cloud storage. This means lots of existing tools work with it.           |

### What We Did

#### Step 1: Created the R2 bucket

In the Cloudflare dashboard, we went to Storage & databases → R2 Object Storage and created a bucket named **revis-audio**. Public access is disabled — files can only be accessed through presigned URLs that your server generates.

#### Step 2: Created an API token

We generated an API token with read/write permission on the revis-audio bucket. This gave us three pieces of information that the server needs:

- **Account ID** — identifies your Cloudflare account
- **Access Key ID** — like a username for R2
- **Secret Access Key** — like a password for R2

## How Audio Flows Through R2

**Uploading:** User picks a file → browser sends it to your server → server checks the user is logged in → server sends the file to R2

**Playing:** User clicks play → server creates a temporary presigned URL for that file → browser streams audio directly from R2 (never through your server)

***Plain English:** Streaming directly from R2 (instead of through your server) means your server doesn't get bogged down when lots of people are listening at once. It's like giving guests directions to the buffet instead of carrying plates to each table yourself.*

## 4. Railway (Hosting)

---

### What Is Railway?

Railway is a platform that runs your app's code on the internet. Without it, your app would only work on your own computer. Railway takes your code from GitHub, builds it into a runnable application, and keeps it running 24/7 at a public URL.

### Key Terms

| Term                         | What It Means                                                                                                                                |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Deployment</b>            | The process of taking your code and making it live on the internet. Each time you push new code, Railway creates a new deployment.           |
| <b>GitHub repo</b>           | Your code lives in a GitHub repository (repo). Railway watches this repo and auto-deploys when you push changes.                             |
| <b>Environment variables</b> | Settings stored on Railway (API keys, passwords, etc.) that your app reads at runtime. They're not in your code, so they stay secret.        |
| <b>Volume</b>                | Persistent storage attached to your app. Without it, any files your app creates (like the database) would be erased every time it redeploys. |
| <b>Nixpacks</b>              | Railway's build system that automatically detects what your app needs (Node.js, npm packages, etc.) and builds it.                           |
| <b>Healthcheck</b>           | Railway pings a URL on your app to make sure it's alive. Ours is /api/health.                                                                |

### What We Did

#### Step 1: Pushed code to GitHub

We created a GitHub repository called **AviouslyAvi/revis-cloud** and pushed all the code there. We used the GitHub CLI (`gh auth login`) to authenticate, since GitHub no longer accepts passwords for HTTPS pushes.

#### Step 2: Created the Railway project

We signed up at [railway.com](https://railway.com) and created a new project. We then connected it to the GitHub repo by installing the Railway GitHub App. This lets Railway automatically detect new code pushes and deploy them.



### Step 3: Added environment variables

We added all 8 configuration values to Railway's Variables tab:

| Variable                     | What It's For                                     |
|------------------------------|---------------------------------------------------|
| <b>NODE_ENV</b>              | Tells the app it's in development mode            |
| <b>CLERK_PUBLISHABLE_KEY</b> | The public Clerk key for login forms              |
| <b>CLERK_SECRET_KEY</b>      | The private Clerk key for server-side auth        |
| <b>R2_ACCOUNT_ID</b>         | Your Cloudflare account identifier                |
| <b>R2_ACCESS_KEY_ID</b>      | R2 "username"                                     |
| <b>R2_SECRET_ACCESS_KEY</b>  | R2 "password"                                     |
| <b>R2_BUCKET_NAME</b>        | Which R2 bucket to use (revis-audio)              |
| <b>DATABASE_PATH</b>         | Where to store the database file (/data/revis.db) |

### Step 4: Enabled public networking

We turned on public networking so Railway would assign a URL to the app. The URL it generated is: **revis-cloud-production.up.railway.app**

### Step 5: Attached a persistent volume

We attached a volume mounted at **/data**. This is critical because the app uses a SQLite database to store project info, track metadata, user data, etc. Without a volume, Railway would erase the database every time it redeploys your app (which happens automatically on every code push).

**Plain English:** A volume is like plugging in an external hard drive. The app's built-in storage gets wiped on every deploy, but the volume persists. Your database lives on the volume so it survives redeployments.

### Step 6: Deployed

We clicked Deploy and Railway: (1) pulled the code from GitHub, (2) detected it was a Node.js app, (3) ran `npm install` to install dependencies, (4) built a Docker container, (5) started the server, (6) ran a healthcheck, and (7) made it live. The whole process took about 90 seconds.

Status: **ACTIVE / Online** — deployment successful!

## 5. Connecting Everything

---

### The Code Changes We Made

Four files were modified to make the Clerk integration work properly:

| File                | What Changed                                                                                                   |
|---------------------|----------------------------------------------------------------------------------------------------------------|
| src/server.js       | Added /api/clerk-key endpoint so the frontend can fetch the Clerk publishable key at runtime                   |
| public/index.html   | Updated the Clerk initialization to fetch the key from /api/clerk-key instead of using a hardcoded placeholder |
| public/sign-in.html | Same change — fetches key from API, then loads the Clerk sign-in widget                                        |
| public/sign-up.html | Same change — fetches key from API, then loads the Clerk sign-up widget                                        |

### How Auto-Deploy Works

Now that Railway is connected to your GitHub repo, the workflow for making changes is:

1. Edit code on your computer
2. Run `git add`, `git commit`, `git push`
3. Railway automatically detects the push and redeploys (takes ~90 seconds)
4. Your live site is updated with zero downtime

**Plain English:** You never have to manually deploy again. Just push code to GitHub and Railway takes care of the rest. It's like having a robot butler that redecorates the house every time you update the blueprints.

## 6. Quick Reference

---

### Your Dashboard Links

| What              | Where                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------------|
| Your live app     | <a href="https://revis-cloud-production.up.railway.app">https://revis-cloud-production.up.railway.app</a> |
| Railway dashboard | <a href="https://railway.com">https://railway.com</a> (project: zealous-celebration)                      |
| Clerk dashboard   | <a href="https://dashboard.clerk.com">https://dashboard.clerk.com</a> (app: Revis-Cloud)                  |
| Cloudflare R2     | <a href="https://dash.cloudflare.com">https://dash.cloudflare.com</a> → R2 → revis-audio                  |
| GitHub repo       | <a href="https://github.com/AviouslyAvi/revis-cloud">https://github.com/AviouslyAvi/revis-cloud</a>       |

### What Each Service Costs

| Service       | Cost                                                                  |
|---------------|-----------------------------------------------------------------------|
| Clerk         | Free for up to 10,000 monthly active users                            |
| Cloudflare R2 | Free for first 10 GB storage + 10 million reads/month                 |
| Railway       | Trial: 30 days or \$5.00 free credit, then ~\$5/month for a small app |
| GitHub        | Free for public/private repos                                         |

### What To Do Next

- **Visit your app** at the Railway URL and sign up as the first user
- **Upload a test track** to make sure R2 storage is working
- **Play it back** to verify audio streaming works
- **When ready for production:** switch Clerk from Development to Production mode (this gives you a real domain instead of the test subdomain)
- **Optional:** add a custom domain in Railway's Settings → Networking

### Glossary of Terms Used

| Term                        | Definition                                                                                                                       |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>API</b>                  | Application Programming Interface — a way for software to talk to other software, like a waiter taking your order to the kitchen |
| <b>API key / token</b>      | A password that lets one piece of software authenticate with another                                                             |
| <b>Bucket</b>               | A container for files in cloud storage (like a folder)                                                                           |
| <b>Deploy</b>               | To put your app on the internet so people can use it                                                                             |
| <b>Docker container</b>     | A lightweight, self-contained package with your app + everything it needs to run                                                 |
| <b>Endpoint</b>             | A specific URL that does something (e.g., /api/health returns "OK")                                                              |
| <b>Environment variable</b> | A configuration value stored on the server, not in code (keeps secrets safe)                                                     |
| <b>Express.js</b>           | A popular framework for building web servers in JavaScript                                                                       |
| <b>Git / GitHub</b>         | Git tracks changes to code; GitHub hosts Git repos online                                                                        |
| <b>Healthcheck</b>          | A periodic "are you alive?" ping to make sure your app is running                                                                |
| <b>JWT</b>                  | JSON Web Token — a secure "wristband" that proves a user is logged in                                                            |
| <b>Middleware</b>           | Code that runs between receiving a request and sending a response (like a security checkpoint)                                   |
| <b>Nixpacks</b>             | Railway's auto-detection build system for apps                                                                                   |
| <b>Node.js</b>              | A runtime that lets you run JavaScript on a server (not just in a browser)                                                       |
| <b>Presigned URL</b>        | A temporary link to a private file that expires after a set time                                                                 |
| <b>SQLite</b>               | A lightweight database stored as a single file (no separate server needed)                                                       |
| <b>Volume</b>               | Persistent storage in cloud hosting that survives redeployments                                                                  |